Remarques:

1) ***Oublier*** les ";", les {}, les begin-end, les types.
2) Ajustement de code est indispensable pour sa correction
3) tester tous les codes présentés comme dex exemples
4) Pour avoir des caractères accentués, ajouter au début du code la ligne: *# -*- coding: utf8 -*-*
5) Pour ajouter des package depuis la ligne de commande : ***pip install*** nom_package
6) Pour mettre à jour **pip** : ***python*** -m ***pip install –upgrade pip***

Questions:

1) **Initialiser une variable** (sans déclaration, ni type)

```
i=0
s=0
c= "a"
```

2) **Entrée/Sortie standard** :

2.1. Affichage sur écran : print "a", 2, 3, "\n", 5

2.2. Lecture du clavier : c=**input**(**"donne c:"**)

3) **Structures itératives (while, for)**:

```
while Condition:
  Instruction
For i in range(N): #pour i=0..N-1
  Instruction

For i in range(j, k): #pour i=j..k-1
  Instruction
```

Question 1: Écrire une boucle **while** qui compte la somme de s=1+..+ 100.
Question 2: Idem avec **for**.

4) **Structure de données** :

- Liste : une séquence d'élément entre []. Exemple : [1, 2, 3, …]
- Une liste vide se déclare : **l=[]**
- **l.append(x)** : ajoute x à la fin de la liste
- **l[i]=x** : mettre x à la position i de la liste l
- **len(l)** : donne la longeur de l
- **del(l[i])** : suppression de la case i
- **l.remove(k)** : suppression du premier k éléments
- **l.pop()** : suppression du dernier élément

Question 3: En une seule fois : Lire du clavier une liste de 10 éléments, ensuite afficher cette liste.
Question 4 : Entrer une liste de 6 éléments, ensuite afficher : Afficher la longueur de cette liste. Remplacer l'une des valeurs, enlever une case et en ajouter une autre, puis affiche à nouveau la liste. Utiliser une boucle pour parcourir les éléments de la liste et les afficher un par un.

5) **Importation packages et fonctions:**

```
import librairie # on va utiliser des fonctions de la librairie
...
librairie.fonction() # on précise à nouveau le nom de la librairie
```

```
                        # avec chaque utilisation d'une fonction de cette
librairie
```

Questions 5 **(exemple de package: random):** sachant que:

```
import random                        # package pour les fonctions d'aléatoire
print(random.randint(1,10))         # un entier entre 1 et 10
print(random.randrange(10))         # un entier entre 0 et 9
print(random.randrange(10,100,2)) # entier pair entre 10 et 99 avec un
pas=2
print(random.choice(liste_valeurs))        # une valeur au hasard
random.shuffle(liste_valeurs)              # mélange aléatoire
```

Ecrire un petit code qui permet de:

- Saisir une liste de 10 notes;
- Afficher l'une de ces notes de manière aléatoire;
- Mélanger aléatoirement la liste et l'afficher une 2ème fois;

TP 2: Time

Exercice :

```
import time     # package pour les fonctions d'aléatoire
t1 = time.clock()# t1=le temps courant du système
```

<u>Questions 1</u> :

Ecrire un code permettant de :

- Saisir un text du clavier;
- Afficher le texte;
- Calculer le nombre d'occurrences du caractère "a" dans ce texte;
- Calculer le temps nécessaire pour ce code

6) <u>Les fonctions et le programme main:</u>

```
def f(par1, par2, ...):
    instructions;
    return résultat1, résultat2, ...
if __name__ == '__main__':
    x1, x2, ...=f(val1, val2, ...)
```

<u>Questions 2</u>: (comparaison entre temps d'exécution de deux tris)

- **tri_sélection** utilisant **index_minimum(t,d,f)** : on récupère le minimum du tableau et on le place dans la première case, on récupère le minimum du tableau privé de la première case et on le place dans la deuxième, etc.
- **tri_insertion** utilisant insertion(e,t,n) : prendre le ième élément et le mettre à sa place dans les i-1premières cases déjà triées;
- Ecrire deux fonctions pour implémenter les deux méthodes de tri;
- ajouter un programme principal qui teste les deux fonctions;
- Comparer le temps d'exécution des deux méthodes.

Exercice : (Fichiers)

1) Changer les entrées/sorties standards :

```
import os, sys
orig_stdout = sys.stdout #save the original standard output device=screen
in standard
f = file('out.txt', 'w') # create a file
sys.stdout = f # make f as the stadard output, so all outputs with print
will be in f

print "je suis dans un fichier"

sys.stdout = orig_stdout # recall the original standard output device
f.close()
```

Question 1 :

- Rediriger les sorties des codes de l'exercice 1 vers un fichier texte : .

2) Manipulation de fichiers textes :

```
f = open('monfichier.dat','w') # 'w' :ouverture du fichier en écriture
                               et il sera écrasé et si il n'existe pas
                               il sera créé, 'r' ouverture en lecture,
                               'a' ouverture en lecture et ajout à la
                               fin sans l'écraser.
f.write("coucou ! ") # on écrit coucou dans le fichier. write accepte
                      seulement des chaînes de caractères
f.write(str(12))     # on convertit 12 en texte, car write connait que
du texte
f.write("\n")        # on passe à la ligne dans le fichier
f.close() # fermeture du fichier
f. read ()# récupérer tout le contenu de f
f.readline()#lire une seule ligne


import os # pour avoir accès au commande DOS
os.mkdir("C:/tests python")# créer un répertoire physique
os.chdir("C:/tests python")#changer le répertoire de travail
```

Question 2 :

- Refaire le même traitement de la question 1.

3) Les fichiers csv: (http://www.chicoree.fr/w/Fichiers_CSV_en_Python)
Un fichier CSV (*Comma-separated values*) est un simple représentation de données sous formes de colonnes et lignes (comme Excel). Exemple :

Prix,	Désignation,	date
9.8,	Tarte aux pommes,	11/11/2011
13.40,	Galette des rois,	12/12/2012
2.45,	Beignet,	13/10/2013

Vous pouvez le réaliser avec Excel (avec extension .csv) en une seule colonne.

Fragment de code :

```
import csv# package nécessaire
fname = "out.csv" # le fichier physique
```

Lecture d'un fichier :

```
file = open(fname, "rb")# ouverture en lecture
try:
  reader = csv.reader(file)# reader est dit iterator
  for line in reader:# afficher ligne par ligne
      print line # vous pouvez afficher une seule colonne row[0],
row[1], …

finally:
   file.close()
```

Ecriture dans un fichier :

```
try:
    writer = csv.writer(file)# Création de l'''écrivain'' CSV.
    writer.writerow(('Prix', 'Désignation'))# Écriture de la ligne
    d'en-tête avec le titre des colonnes.
    # Écriture des quelques données.
    writer.writerow((9.80, 'Tarte aux pommes'))
    writer.writerow(('13.40', 'Galette des rois'))
    writer.writerow((2.45, 'Beignet'))
finally: # Fermeture du fichier source
         file.close()
```

Question 3:

- On propose de calculer la moyenne d'un étudiant à partir de ces notes saisies déjà dans un fichier csv. Chaque ligne doit contenir : intitulé de la matière, note examen, note TD, coefficient examen, coefficient TP, coefficient matière. Il est possible d'utiliser les opérateur **int(string), float(string)** pour transformer string en des valeurs numérique. Penser à utiliser le mot break pour passer la première ligne qui contient les information « string » : matière, note, …

usage du package scipy (windows 10)=> télécharger les packages: whl d'ici:
http://www.lfd.uci.edu/~gohlke/pythonlibs/

installer avec: pip (C:\Python27\Scripts\pip.exe)

```
pip install C:\Python27\numpy-1.11.2rc1+mkl-cp27-cp27m-win32.whl
pip install C:\Python27\scipy-0.18.1-cp27-cp27m-win32.whl
```

Graphes avec Python:

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as sps
```

1. `plt.plot(x,y,color="r",label="ma legende"):` affiche en rouge la courbe affine par morceaux reliant les points d'abscisses x et d'ordonnées y,
2. **`plt.hist(data,normed=1)`** : trace un histogramme pour le tableau data (exemple: `data=[0,7,2,2]`). sur les X: les valeurs x de data distribuées sur classes (10 par défaut. On peut les modifier par bins=nb_classes), et sur les Y: occurrences des x dans data. normed=1 normalise les valeurs de Y (le y max sera égale à 1 et ainsi de suite).
3. `plt.bar(X,Y,0.1,color='red'):` trace un diagramme rouge en bâton de largeur=0.1 qui associent à chaque x dans X, un y dans Y.
4. **plt.scatter(x, y)**: affiche le nuage de points d'abscisses x et d'ordonnée y
5. **stem(x, y)**: affiche des barres verticales en position x et de hauteur y
6. **subplot**: subdivise la fenêtre graphique de façon à y afficher plusieurs graphiques dans la même figure. à mettre avant plt.hist. subplot(nbre de lignes, nbre de colonnes, numéro de la figure). Il y a une condition à respecter : le nombre de lignes multiplié par le nombre de colonnes est supérieur ou égal au nombre de figure. exemple subplot(221).
7. plt.legend(loc='best') : affiche la légende définie plus haut avec label (en position optimale) et plt.title("mon titre") donne un titre à la figure.
8. **plt**.title("mon titre") donne un titre à la figure.

Autres remarques:

1. Pour avoir des valeurs réelles, il faut avoir des chiffres à virgule. Exemple 1.0 est une valeur réelle par contre 1 est entière.
2. π s'obtient avec np.pi,

Exemple 1: essaye le programme suivant:

```
import matplotlib.pyplot as plt # Chargement de la bibliothèque
x = [0, 2, 3, 4, 5, 6] # Liste des abscisses
y = [0.1, 3.9, 9.2, 15.5, 24.8, 36.5] # Liste des ordonnées
plt.plot(x, y) # Construction du graphe
plt.show() # Affichage du graphe
```

Exercice 1 :

1. On souhaite maintenant représenter le graphe de la fonction Y=f(X) sur l'intervalle [0, N]. Il faut donc construire une liste d'abscisses réelles avec un pas suffisamment petit pour que le tracé ne soit pas une suite de segments formant une ligne brisée. On peut utiliser **range(début, fin, pas)** pour définir la liste des abscisses, mais elle ne fonctionne qu'avec des paramètres entiers, il faut donc être astucieux :

   - construire la liste des abscisses par compréhension à l'aide de range ;

   - construire alors la liste des ordonnées par compréhension.

   Donc : on doit avoir la forme suivant par exemple

```
x = [k for k in range(0, N, 1)]
y = [f(x)]
plt.plot(x, y)
plt.show()
```

2. Essaye le programme suivant:

```
import matplotlib.pyplot as plt #importation classique du module matplotlib.pyplot
sous l'alias plt
n=1000 #nombre de points
a=-2
```

```
b=2
X=[a+(b-a)*k/(n-1) for k in range(n)] #des valeurs régulièrement espacées.
Y=[x*x-1 for x in X]
plt.plot(X,Y) #on relie les points (X[i],Y[i]) par des segments: c'est une bonne
approximation du graphe. plt.show() #et on l'affiche !
plt.show()
```